信雅达公司关于黄金交易所

加密机安全应用 API 接口设计

(V1.14)

(客户端部分)

编制：任秋安

杭州信雅达科技有限公司

2018 年 8 月

目 录

1. 客户端 API 接口函数种类 3
2. 客户端 API 接口函数的提供形式 3
3. 客户端 API 函数的详细描述 3
3.1. 设备连接类 API 3
3.1.1 SYD_Connect 3
3.1.2 SYD_Disconnect 4
3.1.3 SYD_Heart_Beat 4
3.1.4 SYD_Connect_Status 5
3.1.5 SYD_ReConnect 5
3.1.6 SYD_Reset 5
3.2. 公钥基础类 API 6
3.2.1 SYD_SM2_Sign_C 6
3.2.2 SYD_SM2_Verify 7
3.2.3 SYD_SM2_Encrypt 7
3.2.4 SYD_SM2_Decrypt_C 8
3.2.5 SYD_SM3_Hash（带公钥的） 8
3.2.6 SYD_SM3_Hash_Nopuk（不带公钥） 9
3.2.7 SYD_SM2_Get_PublicKey_C 9
3.3. 会话密钥协商类 API 10
3.3.1 SYD_SM2_GenSessionKey 10
3.3.2 SYD_SM2_ConfirmSessionKey_C 11
3.3.3 SYD_GenerateRandom 12
3.3.4 SYD_GenerateRandom_Bin 12
3.4. 加密与解密类 API 13
3.4.1 SYD_SM4_Encrypt_Data 13
3.4.2 SYD_SM4_Decrypt_Data 13
3.4.3 SYD_SM4_Data_MAC 14
3.5. 证书类 API 14
3.5.1 SYD_SM2_DetachedSign 14
3.5.2 SYD_SM2_DetachedVerify 15
3.5.3 SYD_Get_X509_CertInfo 16
3.5.4 SYD_Get_X509_CertFile_C 16
4. 其它辅助类 API 17
4.1 SYD_BIN2HEX 17
4.2 SYD_HEX2BIN 17
4.3 SYD_Get_DevInfo 18
4.4 SYD_Get_SysInfo 18
5. 错误码含义 19
6. 其它说明 20

1. 客户端 API 接口函数种类

信雅达公司加密设备向用户提供以下五类应用程序接口（API）函数：

- 设备连接类 API
- 公钥基础类 API
- 会话密钥协商类 API
- 加密解密类 API
- 证书类 API

2. 客户端 API 接口函数的提供形式

信雅达公司加密设备的应用程序接口（API）将以 API 头文件+动态链接库的形式提供给应用开发，本 API 接口适用于 Windows 平台使用。

应用程序接口（API）头文件及链接库定义如下：

■ *头文件：*

SydApi_WIN.h　描述会话密钥协商、加解密、数字签名和证书类服务等应用程序接口(API)原型

■ *链接库：*

SydApi.dll　提供会话密钥协商、加解密、数字签名和证书类服务等应用接口动态库

3. 客户端 API 函数的详细描述

3.1. 设备连接类 API

3.1.1 SYD_Connect

/*

功能：

建立到指定地址和端口号的加密设备的 TCP/IP 连接，返回 TCP/IP 连接句柄。

如果采用短连接，交易结束，要调用 SYD_Disconnect 进行释放连接。

函数原型：

```
int SYD_Connect(
        const   char*       sIp,
        const   int         nPort,
```

```
        const   char*     sPwdStr,//预留，目前不用
        const   int       nTimeOut,
                int*      pSocketFd
        );
```

返回值：0 表示成功；非 0 表示失败；

参数说明：

sIp：　　输入。加密设备的 IP 地址。

nPort：　　输入。加密设备的端口号，默认服务端口 8889。

sPwdStr：　　输入。加密设备访问口令,目前保留不用。

nTimeOut:　　输入。连接与接收的超时时间，单位毫秒。

pSocketFd：　　输出。与加密设备建立的连接句柄。

3.1.2 SYD_Disconnect

/*

功能：**

关闭与加密设备的 TCP/IP 连接。

函数原型：

```
int SYD_Disconnect(
        const   int   nSocketFd
        );
```

返回值：0 表示成功；非 0 表示失败；

参数说明：

nSocketFd：　　输入。与加密设备建立的连接句柄。

3.1.3 SYD_Heart_Beat

/*

功能：**

tcp 连接心跳检测

函数原型：

```
int SYD_Heart_Beat(
        const   int   nSocketFd
        );
```

返回值：0 表示成功；非 0 表示失败；

参数说明：

nSocketFd：　　输入。与加密设备建立的连接句柄。

3.1.4 SYD_Connect_Status

/*
功能：**

tcp 连接状态检测

函数原型：

```
int SYD_Connect_Status(
            const   int   nSocketFd
            );
```

返回值：1 表示连接可用，0 表示连接不可用，-1 表示错误；

参数说明：

nSocketFd：　　输入。与加密设备建立的连接句柄。

3.1.5 SYD_ReConnect

/*
功能：**

判断之前的句柄是否可以正常使用，正常则保持句柄不变，异常则关闭该句柄的连接，

并重新建立到指定地址和端口号的加密设备的 TCP/IP 连接，返回 TCP/IP 连接句柄。

函数原型：

```
int SYD_ReConnect(
            const   char*   sIp,
            const   int     nPort,
            const   int     nTimeOut,
                    int*    pSocketFd
            );
```

返回值：0　　原连接可用，句柄保持不变

1　　原连接不可用,重新建立连接成功，产生新句柄

其他　原连接不可用,重新建立连接不成功，返回错误码

参数说明：

sIp：　　输入。 加密设备的 IP 地址。

nPort：　　输入。 加密设备的端口号，默认服务端口 8889。

nTimeOut:　　输入。连接与接收的超时时间，单位秒。

pSocketFd：　　输出。 与加密设备建立的连接句柄。

3.1.6 SYD_Reset

/*
功能：**

断开输入句柄的连接，并重新建立到指定地址和端口号的加密设备的 TCP/IP 连接，返

回 TCP/IP 连接句柄。

函数原型：

```
int SYD_Reset(
        const   char*   sIp,
        const   int     nPort,
        const   int     nTimeOut,
                int*    pSocketFd
        );
```

返回值：0 表示成功；非 0 表示失败；

参数说明：

sIp：　　输入。 加密设备的 IP 地址。

nPort：　　输入。 加密设备的端口号，默认服务端口 8889。

nTimeOut:　　输入。连接与接收的超时时间，单位秒。

pSocketFd：　输出。　与加密设备建立的连接句柄。

3.2. 公钥基础类 API

3.2.1 SYD_SM2_Sign_C

/*

功能：**

通过使用者 ID 所对应的私钥对指定的原始数据进行数字签名。

函数原型：

```
int SYD_SM2_Sign_C(
        const   int             nSocketFd,
        const   char *          pUserID,
        const   int             nOrgDataType,
        const   unsigned char *  pPubKey,
        const   int             nPubKeyLen,
        const   unsigned char *  pOrgData,
        const   int              nOrgDataSize,
                unsigned char *  pSignData,
                int *           pSignDataSize
        );
```

返回值：0 表示成功；非 0 表示失败；

参数说明：

nSocketFd：　　输入。 与加密设备建立的连接句柄。

pUserID：　　输入。 使用者的 ID，最大长度为 16 位数字或字母，右对齐。

nOrgDataType:　　输入。输入的数据类型，0：HASH 值，1：原始数据

pPubKey：　　输入。公钥串，DER 编码，HEX 格式。仅 OrgDataType=1 有效

nPubKeyLen：　　输入。公钥串的长度。仅 OrgDataType=1 有效

pOrgData：　　输入。 待签名的原始数据。

nOrgDataSize：　　输入。 待签名的原始数据的长度。

pSignData：　　输出。 签名数据。
pSignDataSize：　　输出。 签名数据长度。默认输入值为缓冲区的大小>=72 字节。

3.2.2 SYD_SM2_Verify

```
/*************************************************************
```

功能：

用指定的公钥对指定的原始数据进行数字签名验证。

函数原型：

```
int SYD_SM2_Verify(
        const   int             nSocketFd,
        const   unsigned char * pPubKey,
        const   int             nPubKeyLen,
        const   int             nOrgDataType,
        const   unsigned char*   pOrgData,
        const   int             nOrgDataSize,
        const   unsigned char*   pSignData,
        const   int              nSignDataSize
        );
```

返回值：0 表示验证正确；非 0 表示失败；

参数说明：

nSocketFd：　　输入。与加密设备建立的连接句柄。
pPubKey：　　输入。公钥串，DER 编码，HEX 格式。仅 OrgDataType=1 有效
nPubKeyLen：　　输入。公钥串的长度。仅 OrgDataType=1 有效
nOrgDataType:　　输入。输入的数据类型，0：HASH 值，1：原始数据
pOrgData：　　输入。签名的原始数据。
nOrgDataSize：　　输入。签名的原始数据的长度。
pSignData：　　输入。签名数据。
nSignDataSize：　　输入。签名数据长度。

3.2.3 SYD_SM2_Encrypt

```
/*************************************************************
```

功能：

用指定的 SM2 公钥对指定的原始数据进行加密。

函数原型：

```
int SYD_SM2_Encrypt(
        const   int             nSocketFd,
        const   unsigned char * pPubKey,
        const   int             nPubKeyLen,
        const   unsigned char*   pOrgData,
        const   int             nOrgDataSize,
                unsigned char*   pOutData,
```

```
                int *           pOutDataSize
        );
```

返回值：0 表示成功；非 0 表示失败；

参数说明：

nSocketFd：	输入。与加密设备建立的连接句柄。
pPubKey：	输入。公钥串，DER 编码，HEX 格式。
nPubKeyLen：	输入。公钥串的长度。
pOrgData：	输入。待加密的原始数据。
nOrgDataSize：	输入。待加密的原始数据的长度
pOutData：	输出。密文数据，DER 编码。
pOutDataSize：	输出。密文数据长度指针。 默认输入值为缓冲区的大小>=112+ nOrgDataSize。

3.2.4 SYD_SM2_Decrypt_C

/*

功能：

通过使用者 ID 所对应的私钥对指定的密文数据进行解密并验证。

函数原型：

```
int SYD_SM2_Decrypt_C(
        const   int             nSocketFd,
        const   char *          pUserID,
        const   unsigned char * pOrgData,
        const   int             nOrgDataSize,
                unsigned char * pOutData,
                int *           pOutDataSize
        );
```

返回值：0 表示成功；非 0 表示失败；

参数说明：

nSocketFd：	输入。与加密设备建立的连接句柄。
pUserID：	输入。使用者的 ID，最大长度为 16 位数字或字母，右对齐。
pOrgData：	输入。加密的数据，DER 编码。
nOrgDataSize：	输入。加密的数据的长度
pOutData：	输出。明文数据。
pOutDataSize：	输出。明文数据长度指针，默认输入值为缓冲区的大小>= nOrgDataSize-104。

3.2.5 SYD_SM3_Hash（带公钥的）

/*

功能：

输入指定的原始数据和公钥，通过 SM3 算法计算数据的散列值。

函数原型：

```
int SYD_SM3_Hash(
                const   int                 nSocketFd,
                const   unsigned char * pPubKey,
                const   int                 nPubKeyLen,
                const   unsigned char*   pOrgData,
                const   int                 nOrgDataSize,
                        unsigned char*   pHash
                );
```

返回值：0 表示成功；非 0 表示失败；

参数说明：

nSocketFd：	输入。与加密设备建立的连接句柄。
pPubKey：	输入。公钥串，DER 编码，HEX 格式。
nPubKeyLen：	输入。公钥串的长度。
pOrgData：	输入。原始数据。
nOrgDataSize：	输入。原始数据的长度。
pHash：	输出。HASH 值，32 个 Bytes。

3.2.6 SYD_SM3_Hash_Nopuk（不带公钥）

/*

功能：**

输入原始数据，通过 SM3 算法计算数据的散列值。

函数原型：

```
int SYD_SM3_Hash_Nopuk(
                const   int                 nSocketFd,
                const   unsigned char* pOrgData,
                const   int                 nOrgDataSize,
                        unsigned char* pHash
                );
```

返回值：0 表示成功；非 0 表示失败；

参数说明：

nSocketFd：	输入。与加密设备建立的连接句柄。
pOrgData：	输入。原始数据。
nOrgDataSize：	输入。原始数据的长度。
pHash：	输出。HASH 值，32 个 Bytes。

3.2.7 SYD_SM2_Get_PublicKey_C

/*

功能：

通过使用者 ID 获得使用者的 SM2 公钥(DER 编码)。

函数原型：

```
int SYD_SM2_Get_PublicKey_C (
        const int nSocketFd,
        const char *pUserID,
        const int nKeyType,
        unsigned char *pPubKey,
        int *pPubKeyLen,
        );
```

返回值：0 表示成功；非 0 表示失败；

参数说明：

nSocketFd：　输入。与加密设备建立的连接句柄。

pUserID：　输入。使用者的 ID，最大长度为 16 位数字或字母，右对齐。

nKeyType:　输入。密钥类型，0x00：签名公钥,0x01:加密公钥。

pPubKey：　输出。输出的公钥串，DER 编码，HEX 格式。

*pPubKeyLen：　输出。输出的公钥串的长度>=136。

3.3. 会话密钥协商类 API

3.3.1 SYD_SM2_GenSessionKey

/

功能描述：

此函数用于会话密钥的产生，用于发起端。

加密机产生随机会话密钥，输出两类密文：

1、 公钥加密的密钥密文，用于密钥的传递。

2、 本地 LMK 加密的密钥，用于本地应用数据的加解密。

函数原型：

```
int   SYD_SM2_GenSessionKey(
            const   int                 nSocketFd,
            const   unsigned char * pPubKey,
            const   int                 nPubKeyLen,
                    unsigned char * pCipherKey,
                    int *               pCipherKeyLen,
                    unsigned char * pSessionKey,
                    int *               pSessionKeyLen,
                    unsigned char * pKCV
            );
```

返回值：0 表示成功；非 0 表示失败；

参数说明：

nSocketFd： 输入。与加密设备建立的连接句柄。
pPubKey： 输入。公钥串，DER 编码，HEX 格式。
nPubKeyLen： 输入。公钥串的长度。
pCipherKey： 输出。公钥加密的会话密钥,DER 编码，HEX 格式
pCipherKeyLen： 输出。返回公钥加密的密文长度指针。默认输入值为缓冲区的大小>=248。
pSessionKey： 输出。本地 LMK 加密的会话密钥，HEX 格式
pSessionKeyLen： 输出。返回本地 LMK 加密的会话密钥长度，HEX 格式(不包括第 1 个字符，第一个字符为密钥类型的 ASCII 值)，一般为 33 个 Bytes。默认输入值为缓冲区的大小。
pKCV: 输出。会话密钥的检验值，HEX 格式，长度为 16H。

3.3.2 SYD_SM2_ConfirmSessionKey_C

/***
功能描述：
此函数用于会话密钥的接收，用于接收端。
加密机通过使用者 ID 所对应的私钥解密对方公钥加密的密文，输出本地 LMK 加密的会话密钥，用于本地应用数据的加解密。
函数原型：

```
int   SYD_SM2_ConfirmSessionKey_C(
            const   int                 nSocketFd,
            const   char *              pUserID,
            const   unsigned char *     pCipherKey,
            const   int                 nCipherKeyLen,
                    unsigned char *     pSessionKey,
                    int *               pSessionKeyLen,
                    unsigned char *     pKCV
            );
```

返回值：0 表示成功；非 0 表示失败；
参数说明：
nSocketFd： 输入。与加密设备建立的连接句柄。
pUserID： 输入。使用者的 ID，最大长度为 16 位数字或字母，右对齐。
pCipherKey： 输入。公钥加密的会话密钥,DER 编码，HEX 格式。
nCipherKeyLen： 输入。公钥加密的密文长度
pSessionKey： 输出。本地 LMK 加密的会话密钥，HEX 格式
pSessionKeyLen： 输出。返回本地 LMK 加密的会话密钥长度，HEX 格式(不包括第 1 个字符，第一个字符为密钥类型的 ASCII 值)，一般为 33 个 Bytes。默认输入值为缓冲区的大小。
pKCV: 输出。会话密钥的检验值，HEX 格式，长度为 16H。

3.3.3 SYD_GenerateRandom

```
/*************************************************************
```

功能描述：

产生要求长度的随机数据串，可用于身份认证。

函数原型：

```
int   SYD_GenerateRandom(
                const   int             nSocketFd,
                const   int             nRandomSize,
                        unsigned char * pRandom
                );
```

返回值：0 表示成功；非 0 表示失败；

参数说明：

nSocketFd：　　输入。与加密设备建立的连接句柄。

nRandomSize：　　输入。需要输出的随机串的长度。

pRandom：　　输出。输出的随机串，长度为 nRandomSize*2,HEX 格式。

3.3.4 SYD_GenerateRandom_Bin

```
/*************************************************************
```

功能描述：

产生要求长度的随机二进制编码数据串，可用于身份认证。

函数原型：

```
int   SYD_GenerateRandom_Bin(
                const   int             nSocketFd,
                const   int             nRandomSize,
                        unsigned char *   pRandom
                );
```

返回值：0 表示成功；非 0 表示失败；

参数说明：

nSocketFd：　　输入。与加密设备建立的连接句柄。

nRandomSize：　　输入。需要输出的随机串的长度。

pRandom：　　输出。输出的随机串，长度为 nRandomSize,BIN 格式。

3.4. 加密与解密类 API

3.4.1 SYD_SM4_Encrypt_Data

```
/*************************************************************
```

功能：

用输入的会话密钥，对指定报文数据进行加密处理。

函数原型：

```
int   SYD_SM4_Encrypt_Data(
                const   int                 nSocketFd,
                const   unsigned char *     pSessionKey,
                const   unsigned char *     pInData,
                const   int                 nInDataSize,
                        unsigned char *     pOutData,
                        int *               pOutDataSize,
                );
```

返回值：0 表示成功；非 0 表示失败；

参数说明：

nSocketFd：	输入。与加密设备建立的连接句柄。
pSessionKey：	输入。本地 LMK 加密的会话密钥，HEX 格式。
pInData：	输入。要加密的报文明文。
nInDataSize	输入。报文输入长度，以字节为单位。
pOutData：	输出。输出的报文密文。
pOutDataSize	输出。输出的报文密文长度是 16 的倍数。默认输入值为缓冲区的大小>= nInDataSize+16。

3.4.2 SYD_SM4_Decrypt_Data

```
/*************************************************************
```

功能描述：

用输入的会话密钥，对指定报文数据进行解密处理。

函数原型：

```
int   SYD_SM4_Decrypt_Data(
                const   int                 nSocketFd,
                const   unsigned char * pSessionKey,
                const   unsigned char * pInData,
                const   int                 nInDataSize,
                        unsigned char * pOutData,
                        int *               pOutDataSize,
                );
```

返回值：0 表示成功；非 0 表示失败；

参数说明：

nSocketFd：	输入。与加密设备建立的连接句柄。
pSessionKey：	输入。本地 LMK 加密的会话密钥，HEX 格式
pInData：	输入。要解密的报文密文。
nInDataSize：	输入。报文输入长度，以字节为单位，必须是 8 或 16 的倍数。
pOutData：	输出。输出的报文明文。
pOutDataSize：	输出。输出的报文明文长度。 默认输入值为缓冲区的大小>= nInDataSize。

3.4.3 SYD_SM4_Data_MAC

```
/*************************************************************
```

功能：

用指定的应用密钥，对指定报文数据产生消息认证码 MAC。

函数原型：

```
int   SYD_SM4_Data_MAC(
                const   int                 nSocketFd,
                const   unsigned char * pSessionKey,
                const   unsigned char * pInData,
                const   int                 nInDataSize,
                        unsigned char * pOutMac,
                        int *               pOutMacSize,
                );
```

返回值：0 表示成功；非 0 表示失败；

参数说明：

nSocketFd：	输入。与加密设备建立的连接句柄。
pSessionKey：	输入。本地 LMK 加密的会话密钥，HEX 格式。
pInData：	输入。要加密的报文明文。
nInDataSize：	输入。报文输入长度，以字节为单位。
pOutMac：	输出。输出的报文 MAC。
pOutMacSize：	输出。输出的 MAC 长度，16 字节。

3.5. 证书类 API

3.5.1 SYD_SM2_DetachedSign

```
/*************************************************************
```

功能：

通过指定的私钥对指定的原始数据编制带公钥证书的数字签名（遵循 PKCS#7）。

函数原型：

```
int SYD_SM2_DetachedSign(
                const   int                 nSocketFd,
                const   unsigned char*      pOrgData,
                const   int                 nOrgDataSize,
                const   char *               sCertDN,
                        unsigned char *     pSignData,
                        int *               pSignDataSize
                );
```

返回值：0 表示成功；非 0 表示失败；

参数说明：

nSocketFd：	输入。	与加密设备建立的连接句柄。
pOrgData：	输入。	待签名的原始数据。
nOrgDataSize：	输入。	待签名的原始数据的长度。
sCertDN:	输入。	签名者证书 DN。
pSignData：	输出。	签名数据。
pSignDataSize：	输出。	签名数据长度>=4096。

3.5.2 SYD_SM2_DetachedVerify

/*

功能：**

对指定的原始数据核验其带公钥证书签名。

函数原型：

```
int SYD_SM2_DetachedVerify(
                const   int                 nSocketFd,
                const   unsigned char *     pOrgData,
                const   int                 nOrgDataSize,
                const   unsigned char *     pSignData,
                const   int                 nSignDataSize
                );
```

返回值：0 表示成功；非 0 表示失败；

参数说明：

nSocketFd：	输入。与加密设备建立的连接句柄。
pOrgData：	输入。待签名的原始数据。
nOrgDataSize：	输入。待签名的原始数据的长度。
pSignData：	输入。签名数据。
pSignDataSize：	输入。签名数据长度。

3.5.3 SYD_Get_X509_CertInfo

/***

功能：

输入 X509 格式的证书，获取证书信息。

函数原型：

```
int SYD_Get_X509_CertInfo (
                const   int                 nSocketFd,
                const   unsigned char * pCert,
                const   int                 nCertLen,
                        unsigned char * CertInfo,
                        int*                pCertInfoLen
                );
```

返回值：0 表示成功；非 0 表示失败；

参数说明：

nSocketFd： 输入。与加密设备建立的连接句柄。

pCert: 输入。X509 证书。

nCertLen: 输入。X509 证书长度。

pCertInfo： 输出。证书信息数据。

证书信息结构如下

```
typedef struct
{
    char issuer[257];           /*  颁发者 DN*/
    char serialNumber[65];      /*  证书序列号*/
    char subject[257];          /*  证书主题*/
    char notBefore[15];         /*  证书有效期的起始时间*/
    char notAfter[15];          /*  证书有效期的终止时间*/
    char pubkey[257];
} CERT_INFO;
```

pCertInfoLen： 输出。证书信息数据的长度,默认输入值为缓冲区的大小>=sizeof(CERT_INFO)。

3.5.4 SYD_Get_X509_CertFile_C

/***

功能：

通过使用者 ID 获取的 X509 证书文件。

函数原型：

```
int SYD_Get_X509_CertFile _C(
                const   int                 nSocketFd,
                const   char *               pUserID,
                const   int                 nCertType,
                        unsigned char * pCert,
```

```
                    int *           pCertLen
                );
```

返回值：0 表示成功；非 0 表示失败；

参数说明：

nSocketFd：　　输入。与加密设备建立的连接句柄。

pUserID：　　输入。使用者的 ID，最大长度为 16 位数字或字母(要与生成实体的机构号保持一致)。

nCertType:　　输入。证书类型，0x00:签名证书,0x01:加密证书

pCert：　　输出。证书文件数据,BASE64 编码。

pCertLen：　　输出。输出证书文件长度指针>=4096。

4. 其它辅助类 API

4.1 SYD_BIN2HEX

/*

功能：**

将二进制的数据串转为 HEX 数据串，输出长度为输入长度的 2 倍。

函数原型：

```
int   SYD_BIN2HEX (
                const   unsigned char *   pBin,
                const   int               nBinLen,
                        unsigned char * pHex,
                );
```

返回值：0 表示成功；非 0 表示失败；

参数说明：

pBin：　　输入。二进制数据串。

nBinLen　　输入。二进制数据串长度。

pHex：　　输出。输出的 HEX 格式串，长度为 nBinLen*2。

注意：pHex 需要留出足够的缓冲区。

4.2 SYD_HEX2BIN

/*

功能：**

将 HEX 的数据串转为二制串数据串，输出长度为输入长度的 1/2。

输入长度必须为偶数。

函数原型：

```
int   SYD_HEX2BIN (
```

```
                const   unsigned char *    pHex,
                const   int                nHexLen,
                        unsigned char * pBin,
        );
```

返回值：0 表示成功；非 0 表示失败；

参数说明：

pHex：　　输入。HEX 格式数据串。

nHexLen:　　输入。HEX 格式数据串长度（输入长度必须为偶数）。

pBin：　　输出。输出的 BIN 二进制串，长度为 HexLen/2。

注意：pBin 需要留出足够的缓冲区。

4.3 SYD_Get_DevInfo

/*

功能：

获取加密设备的设备信息。

函数原型：

```
int SYD_Get_DevInfo(
                const   int                nSocketFd,
                        unsigned char *    pDevInfo,
                        int *              pDevInfoLen
                );
```

返回值：0 表示成功；非 0 表示失败；

参数说明：

nSocketFd：　　输入。与加密设备建立的连接句柄。

pDevInfo：　　输出。加密机设备信息结构。

设备信息结构如下

```
typedef struct
{
    char vender[40];        /*  生成厂家 SunYard*/
    char device[16];        /*  设备型号*/
    char serialno[16];      /*  设备序号*/
    char version[10];       /*  软件版本号*/
} DEV_INFO;
```

pDevInfoLen：　　输出。加密机设备信息结构长度>=82。

4.4 SYD_Get_SysInfo

/*

功能：

获取加密设备的系统资源资源信息。

函数原型：

```
int SYD_Get_SysInfo(
                const   int                 nSocketFd,
                        unsigned char *     pSysInfo,
                        int *               pSysInfoLen
                );
```

返回值：0 表示成功；非 0 表示失败；

参数说明：

nSocketFd：　输入。与加密设备建立的连接句柄。

pSysInfo：　输出。加密机设备系统资源信息结构。

系统资源信息结构如下

```
typedef struct
{
    char cpu[8];            /* CPU 可利用率*/
    char mem[8];            /*  内存可利用率*/
    char disk[8];           /*  存储可利用率*/
} SYS_INFO;
```

pSysInfoLen：　输出。加密机设备系统资源信息结构长度>=24。

5. 错误码含义

错 误 代 码 (10 进制)	错 误 代 码 (16 进制)	错误含义
-1	FFFF	Socket 错误
-2	FFFE	对端关闭连接
-3	FFFD	网络超时
-4	FFFC	返回包的序列号有误
-5	FFFB	返回包的命令码有误
-6	FFFA	输出缓冲区预留太小
-25	FFE7	输入的 HASH 值长度应该为 32 个字节
-26	FFE6	用户标识超长(应该<=16)
-27	FFE5	传入数据格式非法(空指针)
-28	FFE4	传入 xx 类型不存在（密钥类型、数据类型）
-29	FFE3	传入长度参数非法
33	0021	此用户标识的密钥不存在
4	0004	DER 编码格式错误
21	0015	输入的数据格式错误
19	0013	数据解密错(C3)
2	0002	签名验证错误
32770	8002	签名验证错误(P7)
36868	9004	DN 所指向的实体不存在
32774	8006	证书或签名数据的格式错

374	0176	系统主密钥丢失
7	0007	交易未授权
128	0080	报文数据长度错误
28679	7007	IP 地址未授权(P7)

错误码说明

十进制错误码为负数的是 API 返回错误码

十进制错误码为正数的是底层返回错误码

-29 错误码情况包括：

公私钥长度不是 136；

解密数据长度不是 8 或 16 的倍数；

会话密钥长度不是 33；

传入参数长度超出范围；

HEX 格式数据串长度不是偶数

6. 其它说明

1. HEX 格式：可见字符，BCD 码的扩展。
2. 会话密钥的协商建议由后端产生，客户端验证确认的方式。
3. 为提高系统的可用性以及可扩展性，加密机上不存储会话密钥。
4. KCV 检验值，密钥校验值，对于两端会话密钥协商，用此值可以检验两端会话密钥的一致性，即同一个会话密钥，其 KCV 是相同的。
5. 为防止缓冲区溢出，API 函数输出数据时会检测预留输出缓冲区的大小。
6. 加密采用 ECB 模式，MAC 采用 CBC 模式，填充模式遵循 PBOC3.0 规范。
7. P1 签名（Sign/Verify 系列）原数据数据最大支持 7168Byte;P7 签名（detachedSign/DetachedVerify 系列）原数据最大支持 4096Byte;SM2 加解密系列最大原数据支持 5120Byte;生成随机数最大支持 999Byte；超过最大限制，返回-29 错误码；